

Atmospheric Scattering in Three JS

Angela Gross // CSCI-441 // Project 1

INTRODUCTION

With respect to computer graphics, programmable shaders are one of the more nebulous and intimidating aspects of it. Shaders give a computer graphics developer the power to do whatever they would like while simultaneously forcing them to shoulder the heavy responsibility of doing so. Deceptively short, shaders can infuriate a novice computer graphics developer and cause them to debug their 10-line program until they could almost recite the program by heart. Although delving into the depths of programmable shaders can be never-ending and frustrating, the results that these programmable shaders can provide are incredible. One of those mind-blowing results is a realistic simulation of the atmospheric scattering of light from the sky and the sun on earth. Inspired by a demonstration of atmospheric scattering implemented with the Three JS framework by a graphics developer, Joshua Koo¹, this paper will demonstrate the implementation of the combination of Mie and Rayleigh scattering in order to simulate a realistic render of an outdoor scene. Since shaders are fairly complex and, in general, any type of physical simulation is also complex, the initial motivation of the project was to simply understand the shader that Joshua Koo created using Three JS. However, with a great deal of time and patience, an improvement upon Koo's shader was accomplished by learning and implementing the real-time atmospheric scattering equations developed by A. J. Preetham² and N. Hoffman³.

With the goal of realism without “sacrificing usability”⁴, Preetham developed equations that made use of aspects of the atmosphere that could either be easily computed or approximated. Although the model only provides results for clear and overcast skies only, (i.e. these equations assume a constant density of particles), the emphasis given on incorporating aerial perspective into a rendering system results in an extremely visually appealing sky. Aerial perspective deals with the “desaturation and color shift of distant objects”⁵. Leonardo Da Vinci himself stressed that painters should be sure to whiten the sky near the horizon and make distant objects bluer than the near ones, and both of these effects are integral to aerial perspective⁶. Preetham asserts that the “visually rich appearance” of the sky is caused by the sunlight being scattered. There are a variety of scattering mechanisms but two types of scattering are focused on: Rayleigh and Mie scattering.

Rayleigh scattering describes the behavior of air molecules scattering and is responsible for the sky being

¹ Koo, Joshua. "Sky Sun Shader." Three.js. http://threejs.org/examples/#webgl_shaders_sky.

² Preetham, A. J., Peter Shirley, and Brian Smits. "A Practical Analytic Model for Daylight." University of Utah.

³ Preetham, A. J., and N. Hoffman. "Rendering Outdoor Light Scattering in Real Time." ATI.

⁴ Preetham, A. J., Peter Shirley, and Brian Smits. "A Practical Analytic Model for Daylight." University of Utah.

⁵ Ibid.

⁶ Da Vinci, L. The Notebooks of Leonardo da Vinci, vol. 1. Dover, 1970.

blue⁷. This is due to the varying of a relevant extinction (or absorption) coefficient in preference of shorter wavelengths, and so blue light (400nm) is scattered a great deal more than red light (700 nm). The reds and oranges seen during sunsets are caused by this preferential scattering as well because the light must traverse through more atmosphere to our eyes. Mie scattering, on the other hand, explains the behavior of scattering due to “haze particles”, or particles that are generally larger and scatter more light than molecules. The scattering of these particles cause the whitening of the sky. Although it is difficult to describe haze particles numerically, Preetham uses turbidity, or the ratio of scattering due to haze particles, as a way to do so. Turbidity grossly simplifies the actual nature of the atmosphere, but it is a good enough parameter to use in the application of computer graphics and produces fairly good results.

Preetham was able to describe aerial perspective due to both Mie and Rayleigh scattering using equations that were tailored to being used in programmable shaders. The relevant equations used in the project define aerial perspective (or the combination of Rayleigh and Mie scattering), $L(s)$, along with calculating the scattering coefficients for both Rayleigh and Mie, B_R and B_M . The following describes the aerial perspective formula:

$$L(s) = L_0\tau + L_{in}$$

Where s is the distance travelled to the eye, L_0 is the light (or color) from a distant object, τ is the extinction factor as that light travels, and L_{in} is the in-scattered light. It is important to note that $L_0\tau$ contributes to the absorption and out-scattering of light while L_{in} contributes to the addition and in-scattering of light. The other formula provided by Preetham for the total scattering coefficient for Rayleigh scattering for molecules is:

$$B_R = \frac{8\pi^3(n^2 - 1)^2}{3N\lambda^4} \left(\frac{6 + 3p_n}{6 - 7p_n} \right)$$

Where n is the refractive index of air, (1.0003 in the visible spectrum), N is the number of molecules per unit volume, (2.545×10^{25}), p_n is the depolarization factor, (0.035 is the standard for air), and λ is the wavelength of light. The total scattering coefficient of Mie scattering for haze particles is:

$$B_M = 0.434c\pi \left(\frac{2\pi}{\lambda} \right)^{v-2} K$$

$$c = (0.6544T - 0.6510) \times 10^{-16}$$

Where c is the concentration factor that varies with turbidity T , v is Junge’s exponent, (4 for the sky model), and K are spectral quantities provided by Preetham that vary with λ ⁸. Although Preetham discusses scattering articulately and provides a great basis to it in his paper, he fails at demonstrating an easy way for the graphics developer to evaluate τ and L_{in} . A presentation PDF developed by Preetham and Hoffman, however, does exactly that along with providing a short and concise explanation and background for the equations.

Extinction, as we stated before, is contributed by the absorption and out-scattering of light. They propose

⁷ Preetham, A. J., Peter Shirley, and Brian Smits. "A Practical Analytic Model for Daylight." University of Utah.

⁸ Ibid pg. 28

that the summation of the Rayleigh and Mie coefficients make up the extinction factor, or:

$$\tau = F_{ex}(s) = e^{-B_{ex}s}$$

$$B_{ex} = B_R + B_M$$

Now they discuss the in-scattering, L_{in} , and they take note that, although in-scattering also comes from the sky and the ground, the equation will only take into consideration the sun's contribution to the in-scattering of light. L_{in} is specified as so:

$$L_{in}(s, \theta) = \frac{B_{sc}(\theta)}{B_{ex}} E_{sun}(1 - F_{ex}(s))$$

Where B_{sc} is the combined angular scattering coefficient for Rayleigh and Mie scattering and E_{sun} is the sun's irradiance. In this context θ is the angle between the viewing and sun direction, but, as we see in the next equations, we're only really concerned about the dot product (or $\cos\theta$) between those two normalized vectors. The phase functions for Rayleigh and Mie scattering are, respectively, the following:

$$f_R(\theta) = \frac{3}{16\pi}(1 + \cos^2\theta)$$

$$f_M = \frac{1 - g^2}{4\pi(1 + g^2 - 2g\cos\theta)^{\frac{3}{2}}}$$

Where the Mie phase function is an approximation given by Henyey-Greenstein⁹. The parameter, $-1 \leq g \leq 1$, ranges from backscattering and forward scattering. The two phase functions along with the scattering coefficients of Rayleigh and Mie define $B_{sc}(\theta)$, or:

$$B_{sc}(\theta) = B_R f_R(\theta) + B_M f_M(\theta)$$

Putting all of this together we have defined Preetham's aerial perspective formula, or:

$$L(s, \theta) = L_0 F_{ex}(s) + L_{in}(s, \theta)$$

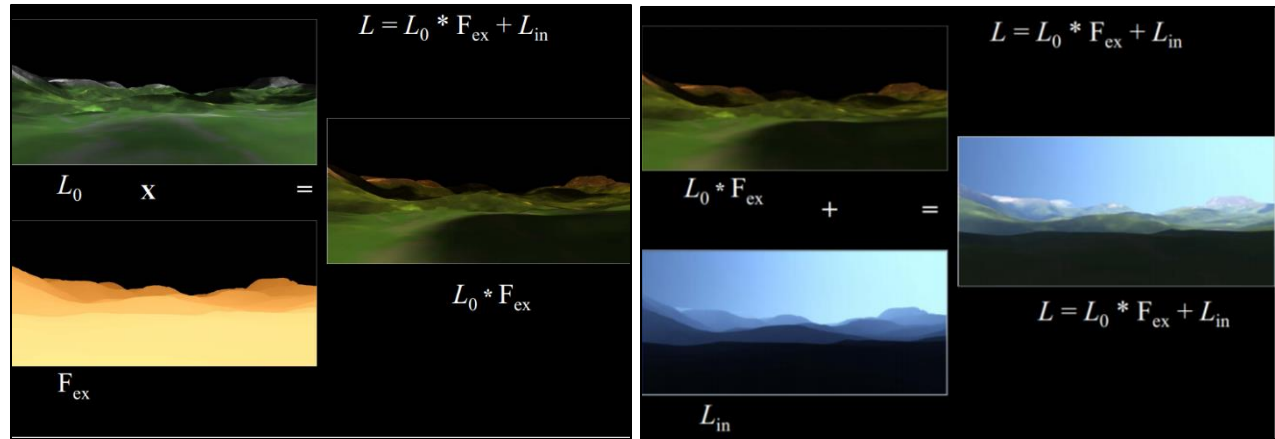
In order to develop a better mental model of what each component of the aerial perspective formula is doing, refer to the graphical representations of the process in Figures 1 and 2. Thus having finally defined and given a sufficient basis for aerial perspective, we can now explore the implementation of these equations as well as our methods of doing so.

METHODS

As suggested earlier on in the paper, the implementation was done within the framework of Three JS, a lightweight JavaScript library and API used to create 3D computer graphics within a web browser¹⁰, along with WebGL, a JavaScript API based on OpenGL for rendering aforementioned 3D graphics within compatible web

⁹ "The Henyey-Greenstein Phase Function." http://www.astro.umd.edu/~jph/HG_note.pdf.

¹⁰ "Three.js." GitHub. <https://github.com/mrdoob/three.js>.



Figures 1 and 2: Combinations of incoming light and extinction along with solar in-scattering. Provided by Preetham and Hoffman.

browsers¹¹. Utilizing a pre-existing Teapot Demo from Eric Haine's Udacity course¹², a 3D scene was first set up by a file called *SkyScene.js*, and then vertex and fragment shaders were added to an object that resides in *SkyShader.js*.

Although implementing *SkyScene.js* is fairly trivial in the context of shader programming, it is still worth mentioning briefly how it functions. A Three JS Perspective Camera and WebGL Renderer are first created along with some camera controls that Haines developed, and Google's dat.GUI is added in order for the user to change the parameters that affect the shader uniform variables. Those are namely turbidity, Rayleigh and Mie coefficients, the g in Mie's phase function, the luminance (which will be discussed a little later), the inclination, the rate that the inclination (and thus the sun's position) changes, and whether or not the camera should follow the sun. The scene is then created, and this consists of adding the camera, updating the sun's position, and adding the sun and sky dome to the scene (which consists of using the aerial perspective shaders to create the dome's material). Finally, the scene is rendered and we cross our fingers and hope that something beautiful appears on the screen. The beauty behind what would appear is, as we alluded before, all due to the programmable shaders that were created in *SkyShader.js*.

SkyShader.js, as mentioned before, is a container for an object that contains the uniform, vertex, and fragment shader variables. The uniforms are defined as stated in the Three JS wiki¹³, and the vertex and fragment shaders are a concatenation of strings that represent the shader program. There are other ways to read in shaders other than doing this (such as using jQuery's Ajax method to read in a shader file's text¹⁴), but these methods are a little more complex and did not seem worth investigating at the time. The uniform variables are parallel to what is in the dat.GUI: a texture 2D sampler to help compute L_0 , the turbidity, Rayleigh and Mie coefficients, the g in Mie's phase function, the luminance, and the sun's xyz position. The vertex shader merely transforms the model

¹¹ "The Khronos Group." WebGL. <https://www.khronos.org/webgl/>.

¹² Haines, Eric. "Interactive 3D Graphics." Course With Three.js & WebGL. <https://www.udacity.com/course/cs291>.

¹³ "Uniform types." GitHub. <https://github.com/mrdoob/three.js/wiki/Uniforms-types>.

¹⁴ "jQuery.ajax()." jQuery API Documentation. <http://api.jquery.com/jquery.ajax/>.

coordinates to view coordinates along with updating the uv coordinates, and the fragment shader uses the equations discussed in the introduction in order to compute the color for the sky dome for each fragment (or pixel). Nearly all of the methods in the fragment shader are simply replications of these scattering equations, and so a discussion about the fundamental differences between Joshua Koo's and Simon Wallner's shader, the parts of this shader that are not directly derived from the given equations, and the improvements made in the shader created for this project will be addressed.

Joshua Koo's fragment shader¹⁵ is heavily inspired by Simon Wallner's shader¹⁶. As opposed to JavaScript, Wallner makes use of C++ to run the 3D scene, but Koo and Wallner both use GLSL to implement these shaders. As such, Koo was able to directly extrapolate the majority of Wallner's shader. The main differences, however, is Koo's use of high dynamic range (HDR) rendering in order to smooth out and interpolate the whites along with doing some interpolation of the extinction values on his own. The HDR rendering from Koo is based upon some work of John Hable, a Naughty Dog developer¹⁷, and some tone mapping done by him from the video game Uncharted 2. Sean O'Neil in his NVIDIA computer graphics book, *GPU Gems*, asserts that "[a]tmospheric scattering doesn't look very good without high-dynamic-range (HDR) rendering ... because these equations can very easily generate images that are too bright or too dark"¹⁸ when referencing Rayleigh and Mie scattering equations. Thus, Koo's use of HDR rendering, the Uncharted 2 tone mapping, and the interpolation of the extinction values help create a much more polished and white-balanced scene.

Along with Koo's white-balancing, there are parts of Wallner's shader that are not explicitly given by Preetham and Hoffman. Unfortunately, they are not completely transparent in *how* they function, but it is known *what* they accomplish. These parts are: computing E_{sun} and the distance light travels, s . Simon computes the intensity, or irradiance, of the sun like so:

$$E_{sun} = 1000.0 \times \max \left(0.0, 1.0 - e^{-\left(\frac{\frac{\pi}{1.95} - \arccos(|sun_{dir}| \cdot |up_{dir}|)}{\frac{3}{2}} \right)} \right)$$

This describes the amount of sky "being revealed" as the intensity of the sun falls off. Wallner (or Koo, for that matter), did not justify or explain the usage of this equation, but it provides a visually appealing sun. The other part that was not transparent, the distance s was described as a series of equations. A separate distance s was given for Mie and Rayleigh scattering based upon the optical length at the zenith for molecules and particles, respectively, and these optical lengths are given by Preetham in another similar paper talking about Mie and Rayleigh scattering¹⁹.

¹⁵ "Three.js." GitHub. <https://github.com/mrdoob/three.js/blob/master/examples/js/SkyShader.js>.

¹⁶ "Kocmoc-demo." GitHub. <https://github.com/SimonWallner/kocmoc-demo/tree/RTVIS/media/shaders>.

¹⁷ Hable, John. "Filmic Tonemapping Operators." Filmic Games » Filmic Tonemapping Operators. <http://filmicgames.com/archives/75>.

¹⁸ O'Neil, Sean. "GPU Gems 2." GPU Gems. http://http.developer.nvidia.com/GPUGems2/gpugems2_chapter16.html.

¹⁹ Preetham, A. J. "Modeling Skylight and Aerial Perspective."

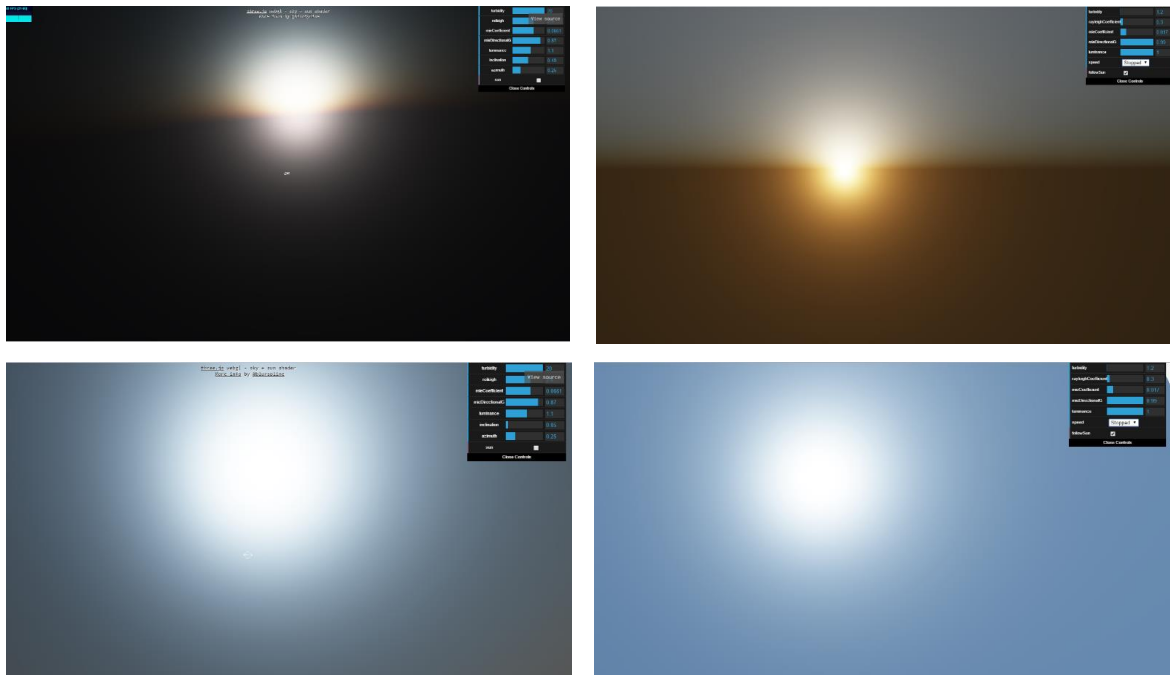
The Rayleigh distance s_R , for example, was computed by Wallner in the following way:

$$\theta_Z = \text{acos}(\max(0.0, |up_{dir}| \cdot |eye_{dir}|))$$

$$s_R = \frac{l_R}{\cos(\theta_Z) + 0.15 \times (93.885 - \text{degrees}(\theta_Z))^{-1.253}}$$

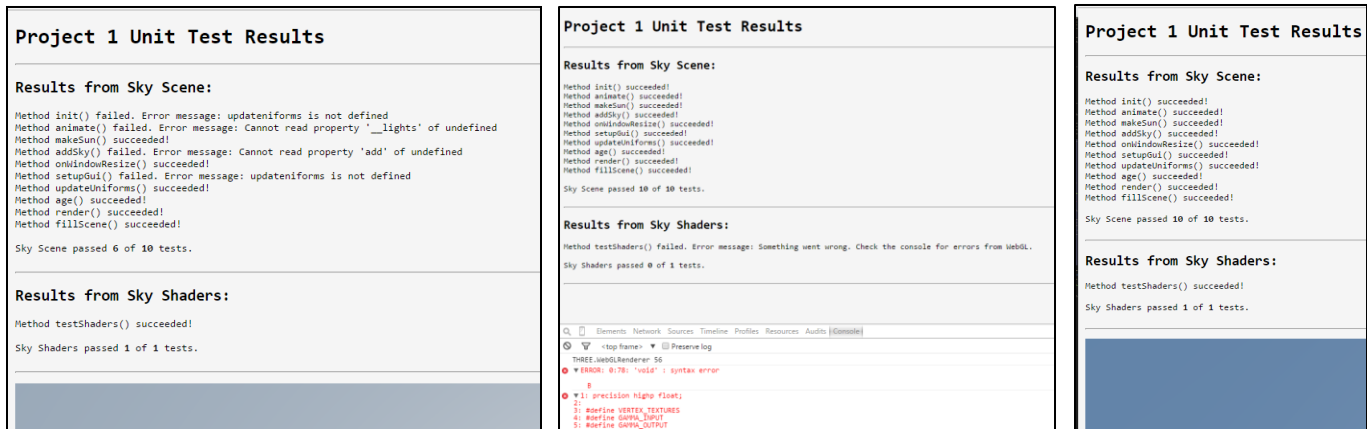
Where θ_Z describes the angle between the up (y-axis) and viewing (or eye) direction and l_R describes the optical length of molecules in the zenith direction. This distance affects the extinction of the lighting model and, as such, helps calculate the scattering done on the earth's surface.

Using all of the equations discussed above, Koo and Wallner developed marvelous fragment shaders using GLSL. However, an integral part of Rayleigh scattering was missing: vibrant reds and oranges at sunsets and vibrant blues at the sun's zenith. At least in the case of Koo's shader, some of this may be due to the fact that he altered the concentration factor in the calculation of the Mie coefficient, raised the in-scattered light to the $\frac{3}{2}$ power along with interpolating the in-scattered light. The shader implemented for this project reverted the concentration factor and the in-scattered light to what was proposed by Preetham, and it also was modified to interpolate between the extinction and the y component of the sun direction as proposed by a fan of Wallner's work²⁰. Both of these changes improved the colors and is shown in Figures 3 – 6, and the reader can run this shader in *index.html*.



Figures 3 - 6: Koo's shader on the left and Gross' shader on the right

²⁰ Wallner, Simon. "Atmospheric Scattering." Simon Wallner. <http://www.simonwallner.at/projects/atmospheric-scattering>.



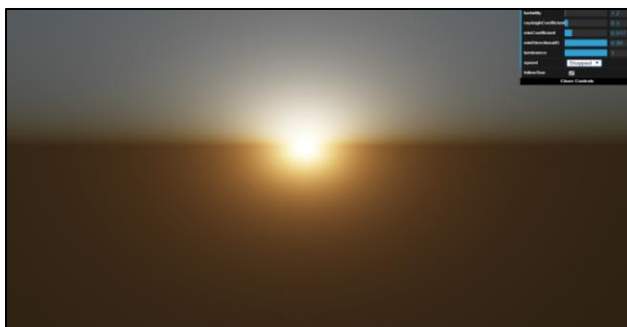
Figures 7 – 9: From left to right: Failed Sky Scene tests, failed Sky Shader tests, and successful tests.

TESTING AND VERIFICATION

In order to make sure that each part created in *SkyScene.js* and *SkyShader.js* is working correctly, a unit testing framework was created to test every method in *SkyScene.js* and compile the shaders in *SkyShader.js*. Since all of the methods did not take anything as arguments or returned any values, wrapping the methods in a try-catch along with looking at the compile-status of WebGL was sufficient to test the files. Examples of the unit tester are provided in Figures 7 – 9, and the reader can use the unit tester themselves by opening up *tests/index.html*.

ANALYSIS

We can further examine the functionality of the shader by investigating the relationships between the important variables in the fragment shader. This can be easily done by using the dat.GUI provided in the shader demo and examining the results visually (as the mathematical relationships have already been established). The basis, or “control image” if you will, of the examination will be the default conditions for each of these variables:



turbidity:1.2
rayleighCoefficient:0.3
mieCoefficient:0.017
mieDirectionalG:0.99
luminance: 1.0
inclination: 0.0

Although we already know what these variables do, the purpose for each can also be found experimentally. The variations of each of these variables (while the others remain unchanged) are visually examined in Figures 10 – 36.

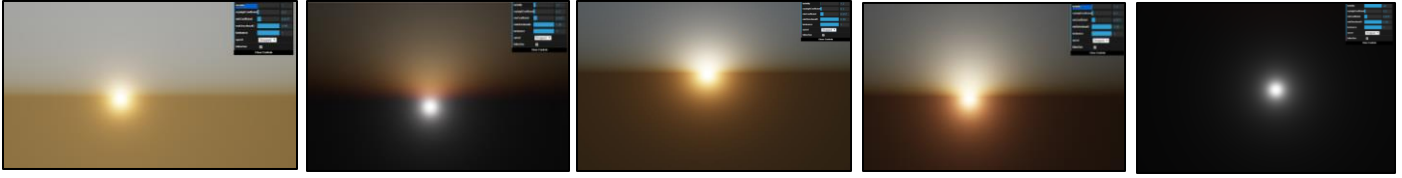
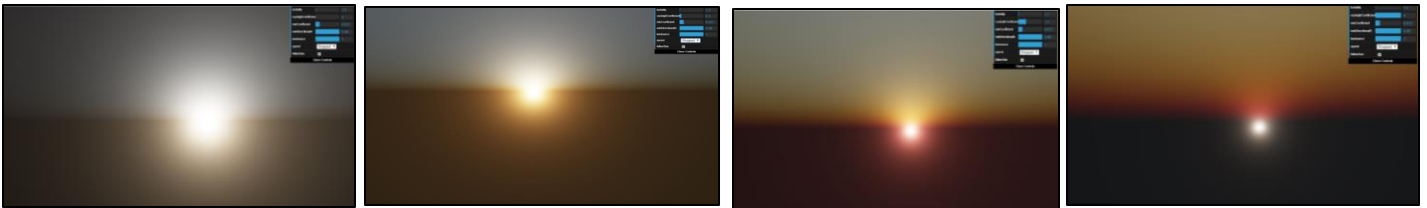
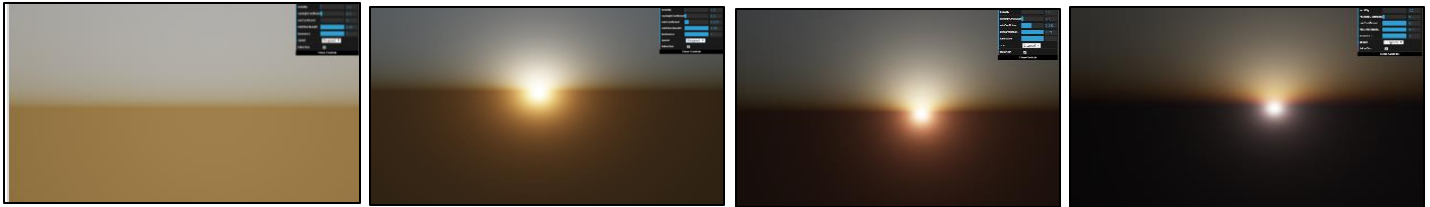


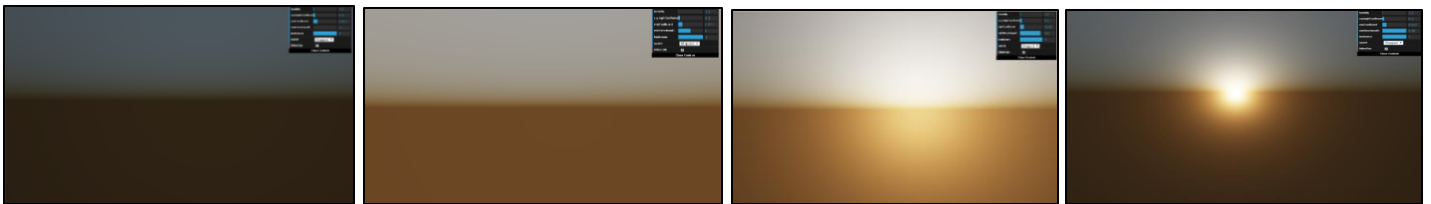
Figure 10 - 14: From left to right: Turbidities 1.0, 1.2, 1.4, 3.1, 7.8, and 20.0



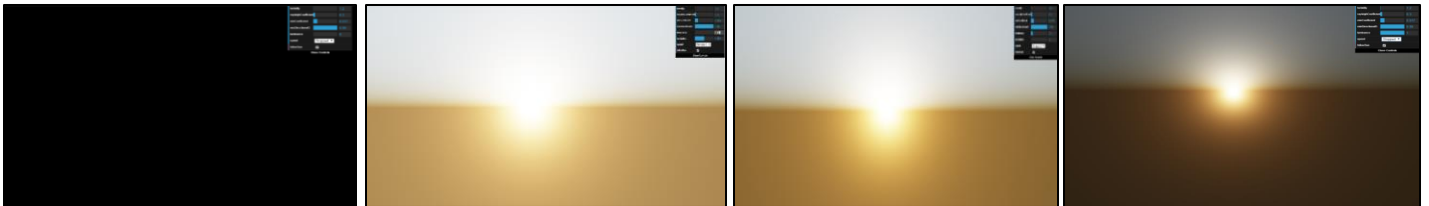
Figures 15-18: From left to right: Rayleigh Coefficients 0.0, 0.3, 1.3, and 4.0



Figures 19-22: From left to right: Mie Coefficients 0.0, 0.017, 0.043, and 0.1



Figures 23-27: From left to right: Mie Directional G's -1.0, 0.0, 0.8, and 0.99



Figures 28-31: From left to right: Luminances 0.0, 0.01, 0.1, and 1.0



Figures 32-36: From left to right: Inclinations -0.0061, 0, 0.0188, 0.1055, and 0.2356

Let's begin with turbidity and the different values for the parameter. The turbidity values, as explained in the introduction, describe how many haze particles are in the atmosphere. This makes sense because, as we can see above, increasing the ratio of haze particles shifts the skies hues toward red and, as we see in Figure 14, the sky is nearly without color. The Rayleigh coefficient is also examined visually with Figures 15 – 18 and, as expected, is also responsible for adding reds along with oranges and yellows. The Mie coefficient also has a similar effect on the color, but it additionally provides the illumination/embodyment of the sun as shown in Figures 19 – 22. The Mie directional G values do not contribute to the red colors, but it does help define the illumination and irradiance of the sun as seen in Figures 23 – 27. The luminance values alter the HDR rendering that Koo added to the shader. Without it, as we see in Figure 28, none of the light will scatter and all will be absorbed. In general, we see that luminance helps with the white-balancing of the scene. Finally, the inclination alters the position (or time, if you will) of the scene, and, at least in this shader, can only be modified when the speed is equal to “Stopped”. As expected, yellows, oranges, and reds appear when the sun is on the horizon due to Rayleigh scattering, and the light shifts toward blue as the sun rises for the same reason.

INTERPRETATION

Expecting to merely have a working knowledge of Joshua Koo's shader, acquiring the ability to improve the shader (even if it was only slight!) was an extremely pleasant surprise. Thus, the results of this project exceeded my expectations. Regardless, I would still like to further investigate and gain a better understanding of the calculations done to compute the sun's irradiance, E_{sun} , the distance that the light travelled, s , and the HDR rendering. Although the project was extraordinarily stressful and, as noted in the introduction, shader programming can be an uphill battle, an immense amount of pride and joy burst from me when this project's shader at long last worked and, quite literally, a “new dawn” began. The ability to create shader programs opens up a whole new world of computer graphics that would not have been possible otherwise. Using the knowledge and tools that I have acquired over the course of this class, this sky shader, and my understanding of shaders and computer graphics in general, will grow, evolve, and improve.

BIBLIOGRAPHY

- Angel, Edward. 2009. *Interactive Computer Graphics: A Top-Down Approach Using OpenGL*. Pearson Education, Inc. Chapter 9: Shaders
- Haines, Eric. n.d. *Interactive3D Graphics*. <https://www.udacity.com/course/cs291>. Lesson 9: Shader Programming
- Hendrickx, Q., R. Smelik, and R. Bidarra. n.d. "An Effective Composition Method for Novice Shader Programmers." *Delft University of Technology*.
- Koo, Joshua. n.d. *Sun Sky Shader*. http://threejs.org/examples/#webgl_shaders_sky.
- Mozilla. n.d. *Lighting in WebGL*. https://developer.mozilla.org/en-US/docs/Web/WebGL/Lighting_in_WebGL.
- mrdoob. n.d. *Three.js*. <https://github.com/mrdoob/three.js>.
- O'Neil, Sean. n.d. *GPU Gems*. http://http.developer.nvidia.com/GPUGems2/gpugems2_chapter16.html.
- Preetham, A. J. n.d. "Modeling Skylight and Aerial Perspective." <http://mathinfo.univ-reims.fr/IMG/pdf/PreethamSig2003CourseNotes.pdf>.
- Preetham, A. J., and N. Hoffman. n.d. "Rendering Outdoor Light Scattering in Real Time." http://www2.ati.com/developer/gdc/GDC02_HoffmanPreetham.pdf.
- Preetham, A. J., Peter Shirley, and Brian Smits. n.d. "A Practical Analytic Model for Daylight." *University of Utah*. <http://www.cs.utah.edu/~shirley/papers/sunsky/sunsky.pdf>.
- n.d. "The Henyey-Greenstein Phase Function." http://www.astro.umd.edu/~jph/HG_note.pdf.
- Wallner, Simon. n.d. *Atmospheric Scattering*. <http://www.simonwallner.at/projects/atmospheric-scattering>.
- Xing, Guanyu, Xuehong Zhou, Qunsheng Peng, Yanli Liu, and Xueying Qin. 2013. "Lighting Simulation of Augmented Outdoor Scene Based on a." *Computer Graphics Forum* 32 (7).